

CSE 4554: Machine Learning

Afrin Jahan Era

ID: 220042132

Department: CSE

Programme: SWE

ML_Lab_Report_02

In this report we analyze implementing linear regression from scratch using a synthetic house price dataset. The dataset includes two features: House Size (sqft, continuous numeric) and Number of Bedrooms (integer, numeric), with the target being House Price (\$1000s).

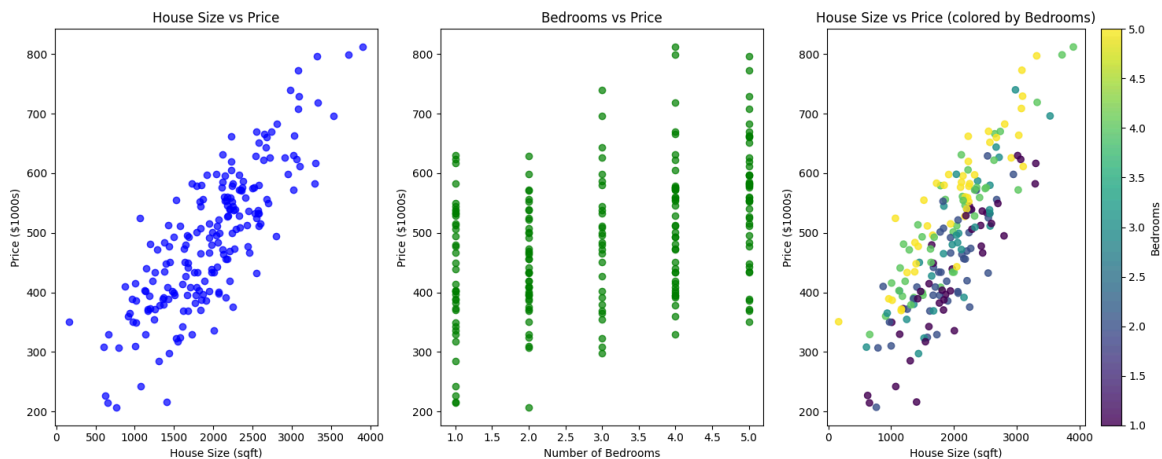
1. Scatter Plots Before and After Normalization

Before Normalization (Original Scale)

The first row of plots shows the raw data:

- House Size vs Price: Strong positive linear relationship. Prices rise clearly with larger size.
- Bedrooms vs Price: Weaker positive trend. More bedrooms increase price, but points overlap a lot.
- House Size vs Price (colored by Bedrooms): Color shows that higher bedroom counts shift points upward for the same size.

The scales are very different. House Size ranges from ~600 to ~3400, while Bedrooms only from 1 to 5.



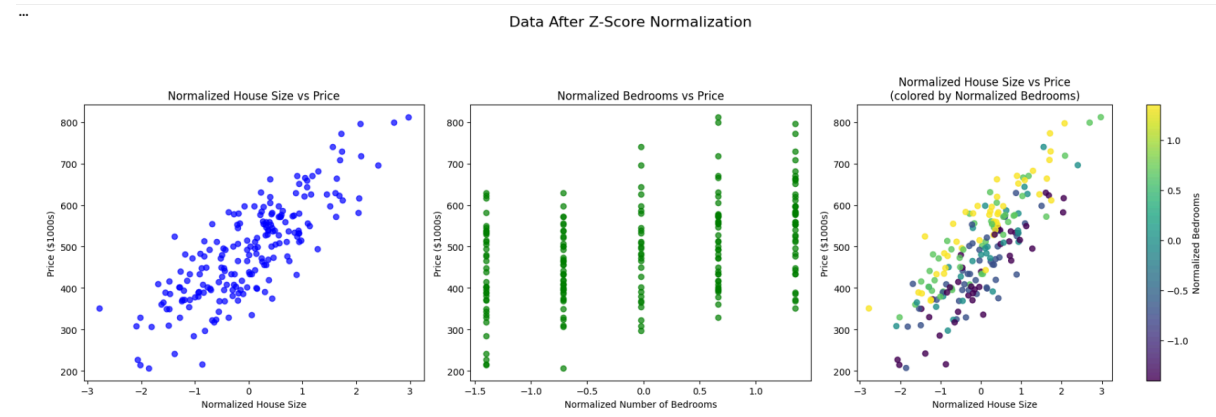
After Normalization (Z-score)

After Z-score normalization, both features have mean 0 and standard deviation 1.

The shape and trends remain exactly the same, but now both axes are on similar scales (roughly -3 to +3).

The best-fit line plot uses normalized House Size (bedrooms fixed at mean 0) and shows the red line passing nicely through the cloud of points.

```
Features normalized using Z-score!  
Mean after normalization: [ 8.88178420e-17 -1.24344979e-16]  
Std after normalization: [1. 1.]
```



Why Normalization is Necessary

House Size has much larger values and variance than Bedrooms. Without normalization, gradient descent would update the size coefficient much more than the bedroom coefficient. This makes convergence slow and unstable. Normalization puts both features on equal footing. It speeds up training, helps the algorithm find the minimum faster, and makes the cost surface more circular.

In this lab, normalization allowed smooth convergence in under 100 epochs.

2. Effect of Learning Rates on Model Convergence

I ran gradient descent for 200 epochs with three learning rates.

The cost vs epoch graph shows clear differences:

- **Learning rate = 0.01 (blue line):** Cost decreases very slowly. Even after 200 epochs, it remains high and continues dropping. Convergence is safe but extremely slow.
- **Learning rate = 0.1 (orange line):** Cost drops sharply in the first 20–30 epochs, then quickly flattens and stays low. This gives fast and stable convergence.
- **Learning rate = 0.5 (green line):** Cost falls the fastest at the beginning and reaches the minimum earliest. It stays flat without any oscillation.

Because features are normalized, even the large rate (0.5) works perfectly.

A good learning rate ensures quick convergence without overshooting. Too small wastes time and too large risks divergence in harder problems.

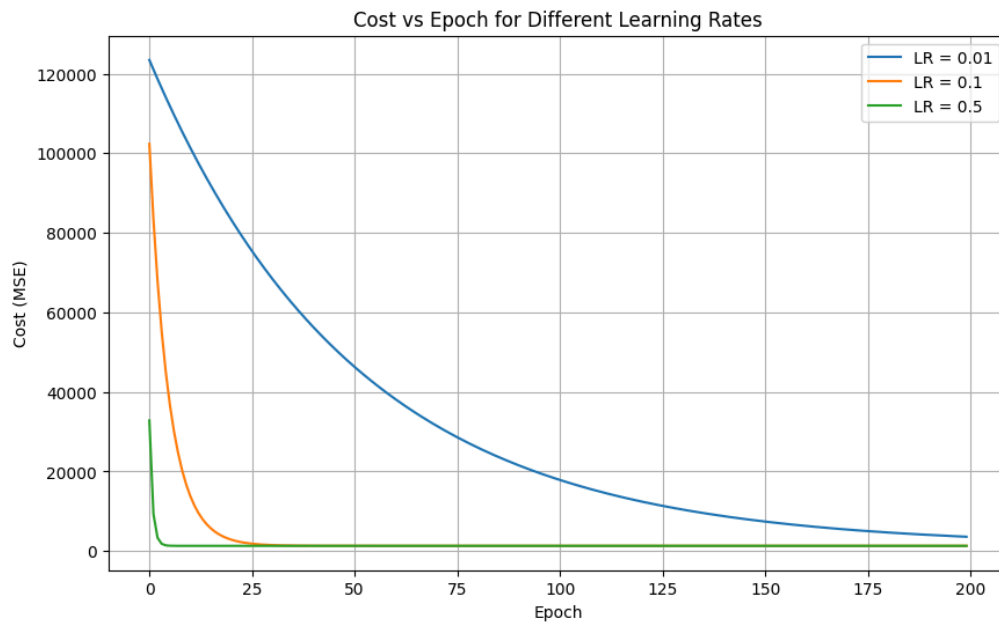
Extended code example:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
for lr in [0.01, 0.1, 0.5]:
    # Small random initialization
    theta_init = np.random.randn(X_train_b.shape[1], 1) * 0.01
    theta, J_history = gradient_descent(X_train_b, y_train, theta_init,
learning_rate=lr, num_epochs=200)
    plt.plot(J_history, label=f'LR = {lr}')

plt.title('Cost vs Epoch for Different Learning Rates')
plt.xlabel('Epoch')
plt.ylabel('Cost (MSE)')
plt.legend()
plt.grid(True)
plt.show()
```

Cost vs Epoch plot:



3. Train vs Test Errors for Different Learning Rates (with Graphs and Theoretical Explanations)

I experimented with three learning rates (0.01, 0.1, and 0.5) over 200 epochs to study their impact on model performance.

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import mean_squared_error

# Dictionaries to store results for plotting
histories = {} # Will store J_history for each LR
results = {} # Will store train_mse and test_mse for each LR

print("Learning Rate Training MSE Test MSE      Final Training Cost (MSE/2)")

for lr in [0.01, 0.1, 0.5]:

    theta_init = np.random.randn(X_train_b.shape[1], 1) * 0.01

    theta_best, J_history = gradient_descent(X_train_b, y_train,
theta_init, learning_rate=lr, num_epochs=200)

    histories[lr] = J_history

    train_pred = X_train_b.dot(theta_best).flatten()
    test_pred = X_test_b.dot(theta_best).flatten()

    train_mse = mean_squared_error(y_train, train_pred)
    test_mse = mean_squared_error(y_test, test_pred)

    results[lr] = {'train_mse': train_mse, 'test_mse': test_mse}

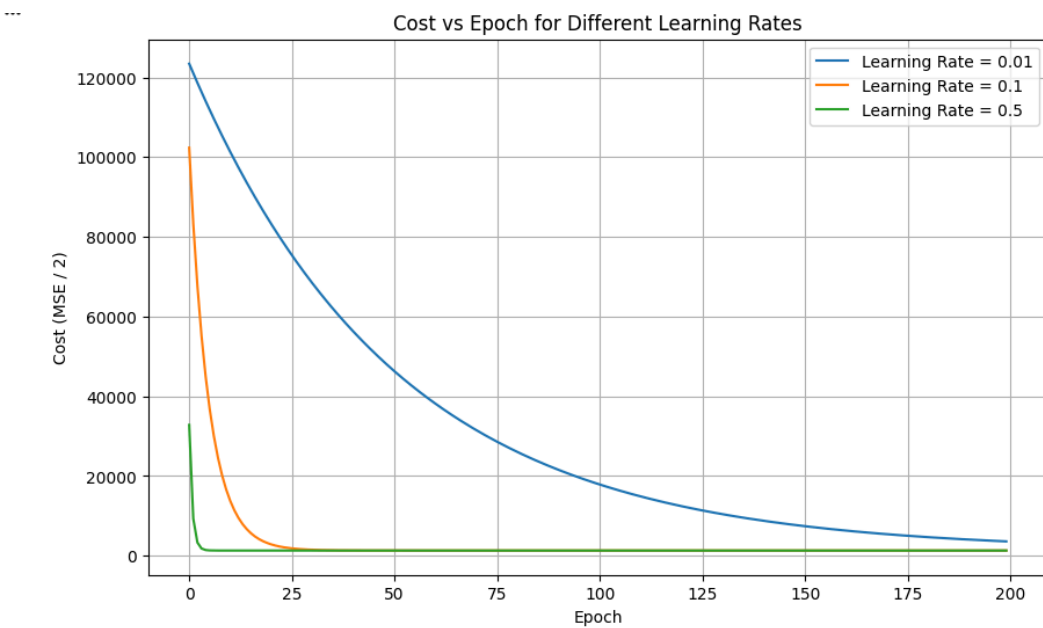
    final_cost = J_history[-1]
    print(f"{lr:<13}  {train_mse:<12.2f}  {test_mse:<12.2f}
{final_cost:<.2f}")
```

Learning Rate	Training MSE	Test MSE	Final Training Cost (MSE/2)
0.01	7208.42	6363.75	3604.21
0.1	2622.48	2263.61	1311.24
0.5	2622.48	2263.61	1311.24

```
# Graph 1: Cost vs Epoch for Different Learning Rates

plt.figure(figsize=(10, 6))
for lr in [0.01, 0.1, 0.5]:
    plt.plot(histories[lr], label=f'Learning Rate = {lr}')

plt.title('Cost vs Epoch for Different Learning Rates')
plt.xlabel('Epoch')
plt.ylabel('Cost (MSE / 2)')
plt.legend()
plt.grid(True)
plt.show()
```



Graph 1: Cost vs Epoch for Different Learning Rates

This line graph explains why the errors differ:

- **Blue line (LR = 0.01):** Cost decreases very slowly and remains high even after 200 epochs. The model never reaches the optimal parameters.
- **Orange line (LR = 0.1):** Cost falls sharply in the first 25–30 epochs and quickly reaches the minimum, staying flat afterward.
- **Green line (LR = 0.5):** Cost drops the fastest and arrives at the same minimum earliest, with no oscillation.

```

# Graph 2: Train vs Test MSE Bar Chart

lrs = [0.01, 0.1, 0.5]
train_mses = [results[lr]['train_mse'] for lr in lrs]
test_mses = [results[lr]['test_mse'] for lr in lrs]

x = np.arange(len(lrs))
width = 0.35

fig, ax = plt.subplots(figsize=(9, 6))
bars1 = ax.bar(x - width/2, train_mses, width, label='Training MSE',
color='skyblue')
bars2 = ax.bar(x + width/2, test_mses, width, label='Test MSE',
color='lightcoral')

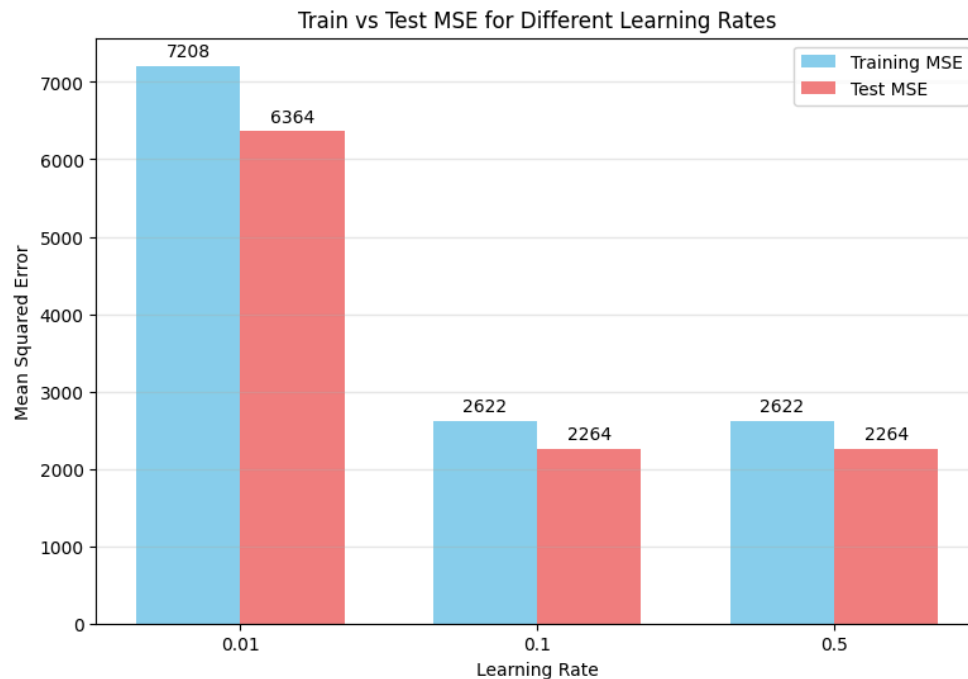
# Adding value labels on top of bars
for bar in bars1 + bars2:
    height = bar.get_height()
    ax.annotate(f'{height:.0f}',
                xy=(bar.get_x() + bar.get_width() / 2, height),
                xytext=(0, 3),
                textcoords="offset points",
                ha='center', va='bottom', fontsize=10)

ax.set_title('Train vs Test MSE for Different Learning Rates')
ax.set_xlabel('Learning Rate')
ax.set_ylabel('Mean Squared Error')
ax.set_xticks(x)
ax.set_xticklabels(lrs)
ax.legend()
ax.grid(True, axis='y', alpha=0.3)

plt.show()

```

..



Graph 2: Train vs Test MSE for Different Learning Rates

This bar chart clearly shows:

- For learning rate 0.01, both training and test MSE are very high (around 7200 and 6360).
- For learning rates 0.1 and 0.5, both errors drop dramatically to the same low values (around 2622 and 2264).

Theoretical Explanation

The learning rate controls the size of steps taken toward the minimum of the cost function.

- **Small learning rate (0.01):** Steps are too tiny. The parameters move slowly down the cost surface. After 200 epochs, the model is still far from the optimal point → severe underfitting → high errors on both training and test data.
- **Appropriate learning rates (0.1 and 0.5):** Steps are large enough to quickly reach the cost minimum. Both achieve the best possible parameters for this dataset → low training MSE (2622) and low test MSE (2264). The small difference between train and test error is expected and indicates excellent generalization (no overfitting).
- Why LR = 0.5 worked without problems: Feature normalization (Z-score) made the cost function smooth and bowl-shaped. This allows larger steps without overshooting or diverging.

4. Handling Non-Numeric Features in Linear Regression

In real datasets, some features may not be pure numbers — for example, house quality rated as "Low", "Medium", "High".

To use such features in linear regression, I would do the following simple steps:

1. Convert categories to numbers I assign ordered numbers to the categories based on their meaning. Example:

Low	1
Middle	2
High	3 This keeps the natural order (higher quality gets a higher number).

2. Normalize the new numbers After converting, the feature becomes numeric. I apply the same Z-score normalization (as in Task 4) to put it on the same scale as other features like House Size and Bedrooms.
3. Train the model exactly the same way I add the bias term, use the same hypothesis function, cost function, and gradient descent. No other changes are needed and the model treats it just like any other feature.
4. Important caution This method works only when the categories have a clear order. If the feature has no order (like - "Location": North, South, East, West), I would create separate yes/no columns for each category (one-hot encoding) instead of assigning numbers.

By converting ordered categories to numbers and normalizing them, I can easily include them in the linear regression model I built from scratch in this lab.